

1.C++入门基础

1. C++课程介绍

C++课程时长：在50+节课，150小时+

C++课程包含哪些部分：C++语法、STL、高阶数据结构

2. C++发展历史

C++的起源可以追溯到1979年，当时Bjarne Stroustrup(本贾尼·斯特劳斯特卢普，这个翻译的名字不同的地方可能有差异)在贝尔实验室从事计算机科学和软件工程的研究工作。面对项目中复杂的软件开发任务，特别是模拟和操作系统的开发工作，他感受到了现有语言（如C语言）在表达能力、可维护性和可扩展性方面的不足。

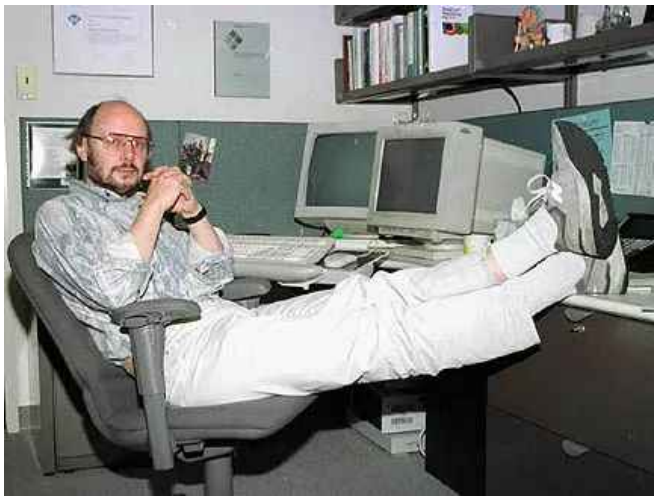
1983年，Bjarne Stroustrup在C语言的基础上添加了面向对象编程的特性，设计出了C++语言的雏形，此时的C++已经有了类、封装、继承等核心概念，为后来的面向对象编程奠定了基础。这一年该语言被正式命名为C++。

在随后的几年中，C++在学术界和工业界的应用逐渐增多。一些大学和研究所开始将C++作为教学和研究的的首选语言，而一些公司也开始在产品开发中尝试使用C++。这一时期，C++的标准库和模板等特性也得到了进一步的完善和发展。

C++的标准化工作于1989年开始，并成立了一个ANSI和ISO（International Standards Organization）国际标准化组织的联合标准化委员会。1994年标准化委员会提出了第一个标准化草案。在该草案中，委员会在保持斯特劳斯特卢普最初定义的所有特征的同时，还增加了部分新特征。

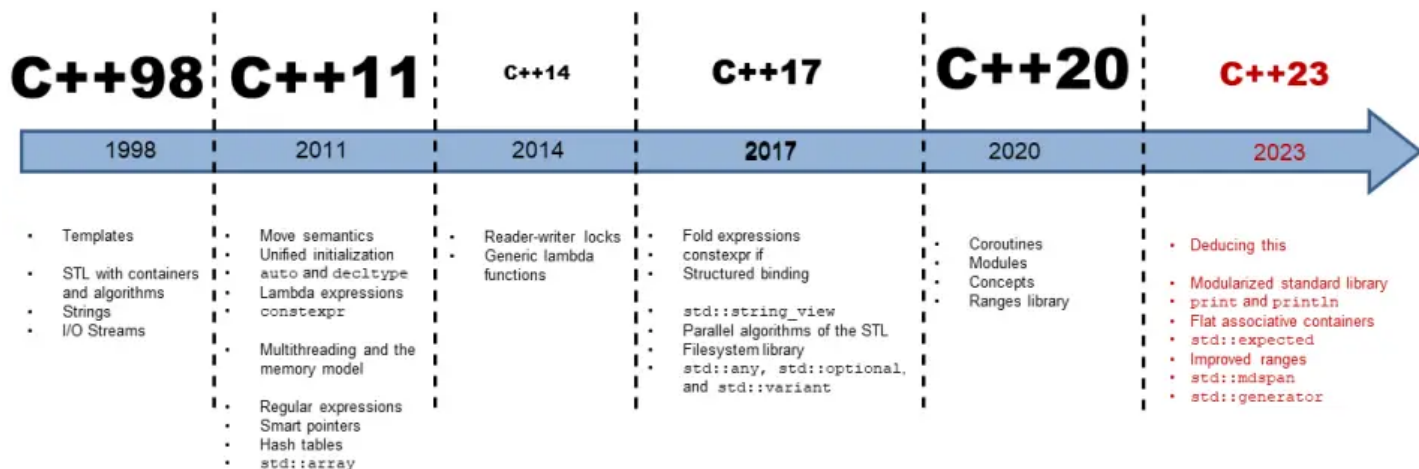
在完成C++标准化的第一个草案后不久，STL（Standard Template Library）是惠普实验室开发的一系列软件的统称。它是由Alexander Stepanov、Meng Lee和David R Musser在惠普实验室工作时所开发出来的。在通过了标准化第一个草案之后，联合标准化委员会投票并通过了将STL包含到C++标准中的提议。STL对C++的扩展超出C++的最初定义范围。虽然在标准中增加STL是个很重要的决定，但也因此延缓了C++标准化的进程。

1997年11月14日，联合标准化委员会通过了该标准的最终草案。1998年，C++的ANSI/ISO标准被投入使用。



2.1 C++版本更新

时间	阶段	内容
1998年	C++98	C++官方第一个版本，绝大多数编译器都支持，得到了国际标准化组织(ISO)和协会认可，以模板方式重写C++标准库，引入了STL(标准模板库)
2003年	C++03	这是C++标准的一个重大修订，主要聚焦于语言的稳定性和兼容性。C++03修正了C++98标准中的错误和漏洞，同时引入了一些新的特性和功能，如tr1库（Technical Report 1）
2011年	C++11	这是一次革命性的更新，增加了大量的新特性和功能，使得C++更像一种新语言。引入了lambda表达式、范围for、右值引用和移动语义、变长模板参数、STL的容器和核心库、智能指针、标准线程库等
2014年	C++14	对C++11的扩展，主要是修复C++11中漏洞以及改进，比如：泛型的lambda表达式、constexpr、constexpr的返回值类型推导，二进制字面常量等
2017年	C++17	C++17进一步增强了C++的功能和表达能力。这次更新引入了if constexpr、折叠表达式等语法特性，同时改进了标准库中的多个组件，如string、filesystem等
2020年	C++20	C++20是C++历史上的又一个重要里程碑。这次更新引入了一系列新特性和改进，如协程（Coroutines）、概念（Concepts）、模块化（Modules）等，为C++的未来发展奠定了坚实的基础。
2023年	C++23	C++23是一个小版本更新，进一步完善和改进现有特性，增加了if constexpr、constexpr、constexpr的返回值类型推导、constexpr的返回值类型推导、constexpr的返回值类型推导等
2026年	C++26	制定中



2.2 关于C++23的一个小故事：

C++一直被诟病的一个地方就是一直没出网络库(networking)，networking之前是在C++23的计划中的，现在C++23已经发布了，但是没有networking，网上引发了一系列的吃瓜和吐槽。中间过程就像发生了宫斗剧一样。



暮无井见铃

往くを継ぎ、来たるを聞く、導き手。

不精确地说，在 C++23 时间片 asio 本来已经在标准化的路上走了一大半，然而一个叫 [Bryce Adelstein Leibach](#) 的男人（以及他的一些盟友）强推 senders/receivers 把这一切都搅黄了。

这里有一篇文章，不排除作者与 B. A. Leibach 有私怨，可以看下。

<https://isocpp.org/files/papers/D2583R0.pdf>

isocpp.org/files/papers/D2583R0.pdf

55 条评论

zaxtyson

networking还推啊，希望2035年能在生产环境下使用🤖

2022-02-03



mci ▸ 张阿大

只能说你的想法有点乐观了

2022-11-16



张阿大

哈哈哈

2022-02-13



Lhgzbxyz

202 人赞同了该文章

距离C++23标准发布已经接近尾声。C++23的制定过程简直跟小说一样带感：

Executors是重中之重，安排了很多场会议，结果因为新冠，大家没法线下开会（照一位委员的说法，上星期开的会，这星期全忘了），迟迟没能讨论好，以极微弱的优势进入January Electronic Poll，但是前景黯淡。受此影响，Networking也确定delay到C++26。

std::generator本来一帆风顺，结果作者突然鸽了，迟迟拿不出R4来。拖了一个月后，果断换作者重新提交paper，这时已经是12月底，按照程序来说是没时间了。因为这玩意实在太重要，LEWG给它上了一个特殊加急通道，目前不能进尚存悬念。

A modular standard library本来意见不一，吵了好久进展不大；危急时刻，作者们果断地把最重要的module std单独写了一篇短小的paper送上去。目前看来是没问题了。

ranges那边东西太多，本来估计要挤掉几个（比如Formatting Ranges），结果LEWG、SG9和SG16三个工作组加班加点把所有都赶出来了，可喜可贺。



南山烟雨珠江潮

互联网大厂程序员，爱好编程，自学转码，欢迎交流。

sender/receiver也没进C++23，两败俱伤，这恐怕是最坏也是最好的结局。

接下来还有3年可以继续互相伤害，C++26再见分晓。

作为吃瓜群众，以后有瓜还是继续吃。

如果想要网络功能继续用asio没问题，asio最大的问题是文档太垃圾，所以曲高和寡。以前的ace，光中文书就有四五本。现在asio连英文书都只有两本，而且都不深入。

在瞬息万变的风潮下，唯有C++的世界属于信息世界的慢生活，多3年对于别的话语可能无法接受，但是对于C++来说稀松平常，因为C++的功能是为以后50年准备的。

再等等呗，如果C++26没有网络，那就等29。至于是不是基于sender/receiver，其实没那么重要。有得用就行。

最后说一下，Bjarne Stroustrup本人对网络功能是相当重视的，所以networking的优先级依然很高。我上次看了最新的《Direction for ISO C++》这个文档，除了对C++以后的发展重点有了一些了解，也能感受到wg21（C++标准委员会）其实是很清醒很谦虚很务实的，从他们的做事态度和清醒的头脑可以感觉到C++未来依然可期。有兴趣的也可以自己看看：

P2452R0 2021 October Library Evolution and Concurrency Networking and Executors Poll Outcomes

C++23的目标

3. C++参考文档：

<https://legacy.cplusplus.com/reference/>

<https://zh.cppreference.com/w/cpp>

<https://en.cppreference.com/w/>

说明：第一个链接不是C++官方文档，标准也只更新到C++11，但是以头文件形式呈现，内容比较易看好懂。后两个链接分别是C++官方文档的中文版和英文版，信息很全，更新到了最新的C++标准，但是相比第一个不那么易看；几个文档各有优势，我们结合着使用。

4. C++的重要性

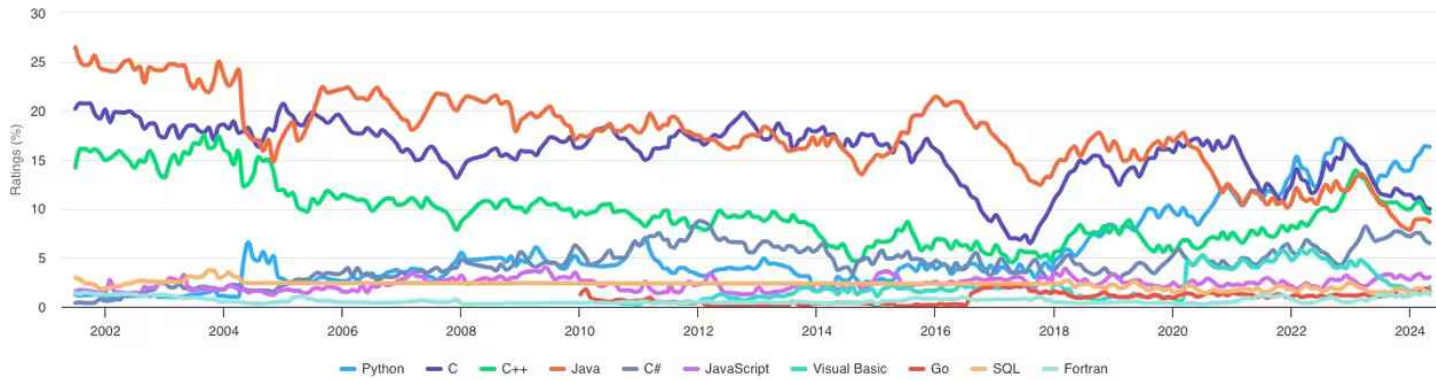
4.1 编程语言排行榜

TIOBE排行榜是根据互联网上有经验的程序员、课程和第三方厂商的数量，并使用搜索引擎（如Google、Bing、Yahoo!）以及Wikipedia、Amazon、YouTube和Baidu（百度）统计出排名数据，只是反映某个编程语言的热门程度，并不能说明一门编程语言好不好，或者一门语言所编写的代码数量多少。

2024年6月TIOBE发布的编程语言排行榜

Jun 2024	Jun 2023	Change	Programming Language		Ratings	Change
1	1			Python	15.39%	+2.93%
2	3	▲		C++	10.03%	-1.33%
3	2	▼		C	9.23%	-3.14%
4	4			Java	8.40%	-2.88%
5	5			C#	6.65%	-0.06%
6	7	▲		JavaScript	3.32%	+0.51%
7	14	▲▲		Go	1.93%	+0.93%
8	9	▲		SQL	1.75%	+0.28%
9	6	▼		Visual Basic	1.66%	-1.67%
10	15	▲▲		Fortran	1.53%	+0.53%
11	11			Delphi/Object Pascal	1.52%	+0.27%
12	19	▲▲		Swift	1.27%	+0.33%
13	10	▼		Assembly language	1.26%	-0.03%
14	12	▼		MATLAB	1.26%	+0.14%
15	8	▼▼		PHP	1.22%	-0.52%
16	13	▼		Scratch	1.17%	+0.15%
17	20	▲		Rust	1.17%	+0.26%
18	18			Ruby	1.11%	+0.17%
19	29	▲▲		Kotlin	1.01%	+0.50%
20	22	▲		COBOL	0.96%	+0.22%

CSDN @逆境清醒



4.2 C++在工作领域中的应用

C++的应用领域服务器端、游戏（引擎）、机器学习引擎、音视频处理、嵌入式软件、电信设备、金融应用、基础库、操作系统、编译器、基础架构、基础工具、硬件交互等很多方面都有。

1. 大型系统软件开发。如编译器、数据库、操作系统、浏览器等等

- 2. 音视频处理。常见的音视频开源库和方案有FFmpeg、WebRTC、Mediasoup、ijkplayer，音视频开发最主要的技术栈就是C++。
- 3. PC客户端开发。一般是开发Windows上的桌面软件，比如WPS之类的，技术栈的话一般是C++和QT，QT 是一个跨平台的 C++图形用户界面（Graphical User Interface，GUI）程序。
- 4. 服务端开发。各种大型应用网络连接的高并发后台服务。这块Java也比较多，C++主要用于一些对性能要求比较高的地方。如：游戏服务、流媒体服务、量化高频交易服务等
- 5. 游戏引擎开发。很多游戏引擎就都是使用C++开发的，游戏开发要掌握C++基础和数据结构，学习图形学知识，掌握游戏引擎和框架，了解引擎实现，引擎源代码可以学习UE4、Cocos2d-x等开源引擎实现
- 6. 嵌入式开发。嵌入式把具有计算能力的主控板嵌入到机器装置或者电子装置的内部，通过软件能够控制这些装置。比如：智能手环、摄像头、扫地机器人、智能音响、门禁系统、车载系统等等，粗略一点，嵌入式开发主要分为嵌入式应用和嵌入式驱动开发。
- 7. 机器学习引擎。机器学习底层的很多算法都是用C++实现的，上层用python封装起来。如果你只想准备数据训练模型，那么学会Python基本上就够了，如果你想做机器学习系统的开发，那么需要学会C++。
- 8. 测试开发/测试。每个公司研发团队，有研发就有测试，测试主要分为测试开发和功能测试，测试开发一般是使用一些测试工具(selenium、Jmeter等)，设计测试用例，然后写一些脚本进行自动化测试，性能测试等，有些还需要自行开发一些测试用具。功能测试主要是根据产品的功能，设计测试用例，然后手动的方式进行测试。

6:36

数据库开发工程师

数据库...工程师

四女·雁塔区·电子城

经验不限

本科

侯女士

北京海量数据 · 招聘经理

今日回复10+次

职位详情

C++ C语言 Linux开发/部署经验

岗位职责：
1、数据库内核研发工作，包括存储引擎、SQL引擎等新特性开发
2、协助架构师完成技术调研和选型工作
招聘要求：
1、计算机相关本科或以上学历
2、为人正直负责，具有良好的沟通能力和团队精神
3、熟悉Linux系统操作，扎实的C/C++基础，熟悉C/C++开发，熟练掌握数据结构和常用算法
4、对技术有强烈的兴趣，喜欢钻研，有好奇心，有较强的学习能力

立即沟通

6:35

编译器开发工程师

编译器...工程师

脚本开发，
能够对输出的性能数据，进行初步分析
后续表现良好，可以转向编译器优化方向

该岗位接受能够长期实习(半年以上),短期实习不接受，且每周能够在职至少四天

要求：
1：能够熟练掌握C/C++ 以及常用数据结构算法
2：具有耐心，能够细致认真分析问题，动手能力强，善于从细节中分析问题
3：对计算机体系结构较为了解。
4：对性能分析较为感兴趣。
5：了解异构编译，对CUDA/opencv/hip等异构编程模型了解更佳。
6：熟悉Linux操作系统，能够独立工作。
7：了解编译器常用优化手段属于加分项。
8：喜欢benchmark研究领域，阅读过对最新相关领域论文更佳。

工作地点可以选择（南京，西安，青岛）

曙光信息产业（北京）有...

立即沟通

6:50

C++软...实习生)

C++ C语言 Qt项目开发经验

2周内活跃

职位详情

C++ C语言 Qt项目开发经验

音视频相关开发经验 Linux开发/部署经验

此岗位面向23届毕业生，实习期间表现优异可转正式。
岗位职责：
1.负责Linux平台应用程序的开发及维护工作。
2.参与需求讨论并完成负责模块功能设计和开发。
要求：
1.计算机或相关专业本科及以上学历。
2.掌握C/C++语言编程，理解算法、数据结构与网络相关知识。
3.掌握音视频基本格式和相关编解码知识。
4.了解TCP/IP、HTTP等网络协议基础知识，熟悉Socket编程。
5.具有QT开发经验或图形可视化相关开发经验经验者优先。

立即沟通

6:52

音视频S...工程师

汪先生

bilibili · 人事专员HR

今日回复10+次

职位详情

C++ 播放器 播放内核 质量

岗位职责：
1、负责/参与 B站音视频播放SDK设计和开发，包括但不限于直播、点播、短视频等；
2、负责/参与 B站音视频播放SDK性能优化和技术演进

岗位要求：
1、本科及以上学历
2、精通C++编程，熟悉C++ 标准特性
3、熟悉常用数据结构，拥有较好程序架构设计思路
4、担任过一定规模C++项目的主程
5、对于编程有充足的热情

立即沟通

6:17

C++开发...西安

职位详情

C++C#QTMySQL

架构设计经验Linux开发/部署经验

数据库开发经验

毕业时间:2024年
招聘截止日期:2025.09.30
岗位职责:
1、软件开发:理解需求文档;完成软件功能开发;编制开发文档;
2、软件维护:单元测试;软硬件联调;现场问题修改;
3、领导安排的其它工作;

任职条件:
1、本科及以上学历,计算机相关专业;
2、熟练掌握 C++ 语言 (C++11以上),熟悉 QT 或其它开发框架;
3、热爱编程,工作中有总结、分享的习惯。

立即沟通

7:01

子公司---发岗位

职位详情

数据结构音视频开发算法基础

WindowsQT

工作职责
1、负责腾讯云会议Windows客户端相关业务需求开发
2、负责视频会议业务开发;
3、负责视频会议稳定性及性能优化;
4、负责视频会议客户反馈问题跟进及效率工具开发;
工作要求
1.本科及以上学历,计算机或软件工程专业;
2.2年以上Windows开发经验,计算机基础扎实,熟悉主流UI框架(QT);
3.熟悉C++语言,具备良好的编码风格,熟练代码管理工具git;
4.具备扎实的数据结构和算法基础;
5.具备良好的分析解决问题能力,良好的学习和分享习惯,工作积极主动,责任心强;
6.有跨平台开发经验或音视频... 查看更多

立即沟通

6:25

领跑X计划-C/C++开发工程师

18-26K·13薪

校招

广州·天河区·棠下在校/应届本科

李妍
深信服科技·招聘经理
今日回复10+次

职位详情

C++MySQL

毕业时间:2025年
招聘截止日期:2024.07.31
工作地点:
深圳市(80%在深圳)、长沙市、北京市、成都市

岗位描述:
负责深信服的云计算、网络安全产品的软件开发。为众多的企业级用户提供更简

立即沟通

6:25

领跑X计划-C/C++开发工程师

18-26K·13薪

校招

广州·天河区·棠下在校/应届本科

李妍
深信服科技·招聘经理
今日回复10+次

职位详情

C++MySQL

毕业时间:2025年
招聘截止日期:2024.07.31
工作地点:
深圳市(80%在深圳)、长沙市、北京市、成都市

岗位描述:
负责深信服的云计算、网络安全产品的软件开发。为众多的企业级用户提供更简

立即沟通

7:08

后台开发

职位详情

功能测试性能测试自动化测试

实习生招聘
[职位描述]
1、负责淘菜菜业务的产品线质量保障工作。
2、担任业务产品测试负责人,有效保证产品线质量
3、参与产品线特色化的测试框架或测试工具研发强的业务分析能力,较好的沟通表达和综合协调能力。
[职位要求]
1.制定测试计划和测试策略,对各个项目关键点进行把关,有效保证产品线质量,参与构建互联网自动化测试平台、性能测试平台、测试环境管理平台,持续集成平台,参与产品线特色化的测试框架或测试工具研发;
2.熟悉Linux操作系统,熟练使用Shell,有互联网测试经验者优先;
3.具备一定的编程能力,熟练掌握Java/C/C++或各类脚本语言中一种;
4.熟悉测试理论与方法,对互联网质量保证领域有强烈的兴趣。

立即沟通

6:33

测试开发

职位详情

功能测试性能测试自动化测试

实习生招聘
[职位描述]
1、负责淘菜菜业务的产品线质量保障工作。
2、担任业务产品测试负责人,有效保证产品线质量
3、参与产品线特色化的测试框架或测试工具研发强的业务分析能力,较好的沟通表达和综合协调能力。
[职位要求]
1.制定测试计划和测试策略,对各个项目关键点进行把关,有效保证产品线质量,参与构建互联网自动化测试平台、性能测试平台、测试环境管理平台,持续集成平台,参与产品线特色化的测试框架或测试工具研发;
2.熟悉Linux操作系统,熟练使用Shell,有互联网测试经验者优先;
3.具备一定的编程能力,熟练掌握Java/C/C++或各类脚本语言中一种;
4.熟悉测试理论与方法,对互联网质量保证领域有强烈的兴趣。

立即沟通

6:22

Unity开...实习岗)

职位详情

C#游戏引擎

岗位职责:
1、使用Unity3D或UE4引擎,进行PC/VR/AR端项目开发;
2、熟练各种数据结构和设计模式的应用,能够独立进行项目开发,保证代码高效;
3、编写项所需的软件设计方案、技术研究文档、功能开发说明文档等文件。
任职要求:
1、本科及以上学历,计算机相关专业;
2、至少熟悉并能熟练使用C#,C,C++语言中的一种。
3、具有独立分析问题和解决问题的能力,以及较强的学习能力;
4、善于语言表达,善于与团队协作;
优先项:
1、有虚拟仿真、教学培训类项目经验者优先。
2、擅长拟写技术方案等文档者优先。
3、熟悉Unity或UE4技术架构优先。

立即沟通

6:21

嵌入式工程师

薪资范围:本科7000-10000/月*14 硕士8000-11000/月*14
岗位职责:
1、参与项目需求分析,产品功能详细设计
2、在主管的指导下进行嵌入式应用软件开发。
3、协助主管进行外设驱动软件的开发。
4、上级主管交办的其他工作。

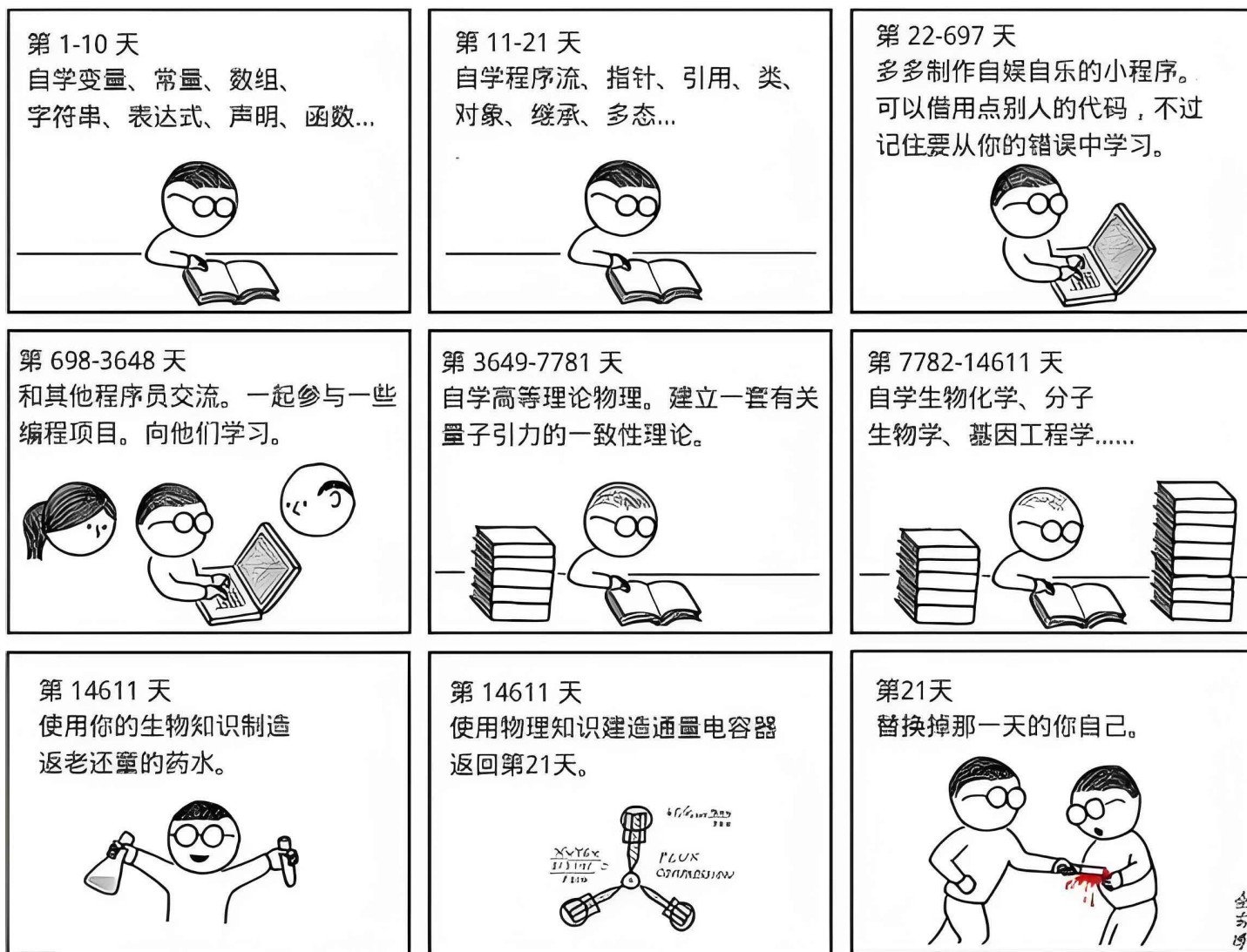
任职要求:
1.本科或本科以上学历
2.计算机软件类、电子信息类等相关专业
3.这是一个校招岗位,但候选人应当熟练掌握C/C++语言编程
4.候选人应当对操作系统有一定的了解,如果有Linux应用软件/驱动软件的开发经验是加分项
5.我们希望你有较强的沟通协作能力,是一个团队工作者
6.我们希望你对待工作有积极态度,能够坚守职业准则,有较强烈的责任意识
7.我们希望你遇到工作中的困难,能有话直说,就事论事
薪酬福利:
五险一金、带薪年假、节日福利、餐补、房补、交通补贴、体检、团建、培训、晋升通道

立即沟通

5. C++学习建议和书籍推荐

5.1 C++学习难度

首先第一个问题,C++难学吗?首先在这里告诉大家C++是一个相对难学难精的语言,相比其他一些语言,学习难度要高一些要陡峭一些,这里有历史包袱的问题,也有语言本身设计和发展历史的问题。网上以前一直流传下面这个21天内自学精通C++的梗。当然自学难度很大,跟着比特课程走,相对会好一些。

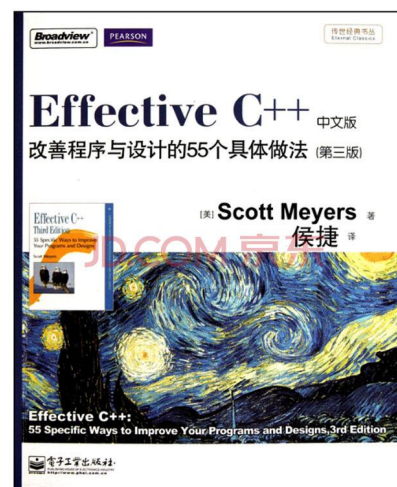
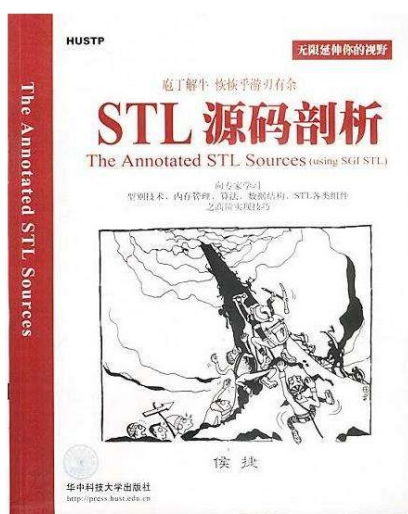
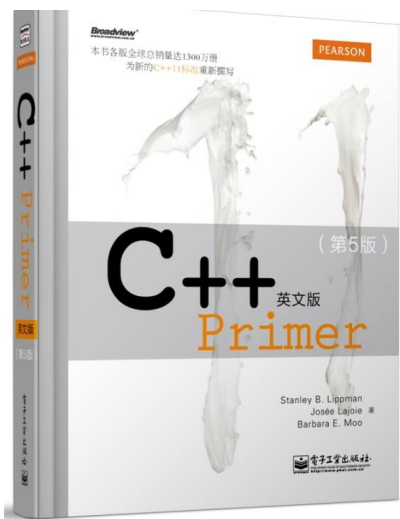


据我所知，这是一种最简单的办法，可以
“在21天内自学 C++”。

5.2 C++学习建议

1. 建议每节课下来把课堂讲过的课堂样例都要练习一遍，理解对应的知识。
2. 建议如果时间能跟上最好每节课总结博客或笔记，如果时间少，至少重点章节整理笔记。

5.3 学习书籍推荐



C++ Primer: 主要讲解语法，经典的语法书籍，前后中期都可以看，前期如果自学看可能会有点晦涩难懂，能看懂多少看懂多少，就当预习，学了比特课程后，中后期作为语法字典，非常好用。

STL源码剖析: 主要从底层实现的角度结合STL源码，庖丁解牛式剖析STL的实现，是侯捷老师的经典之作。可以很好的帮助我们学习别人用语法是如何实现出高效简洁的数据结构和算法代码，如何使用泛型封装等。让我们不再坐井观天，闭门造车，本书课程上一半以后，中后期可以看。

Effective C++: 本书也是侯捷老师翻译的，本书有的一句评价，把C++程序员分为看过此书的和没看过此书的。本书主要讲了55个如何正确高效使用C++的条款，建议中后期可以看一遍，工作1-2年后再看一遍，相信会有不一样的收获。

6. C++的第一个程序

C++兼容C语言绝大多数的语法，所以C语言实现的hello world依旧可以运行，C++中需要把定义文件代码后缀改为.cpp，vs编译器看到是.cpp就会调用C++编译器编译，linux下要用g++编译，不再是gcc

```
1 // test.cpp
2 #include<stdio.h>
3
4 int main()
5 {
6     printf("hello world\n");
7
8     return 0;
9 }
```

当然C++有一套自己的输入输出，严格说C++版本的hello world应该是这样写的。

```
1 // test.cpp
2 // 这里的std cout等我们都看不懂，没关系，下面我们会依次讲解
3 #include<iostream>
4 using namespace std;
```

```

5
6 int main()
7 {
8     cout << "hello world\n" << endl;
9
10    return 0;
11 }

```

7. 命名空间

7.1 namespace的价值

在C/C++中，变量、函数和后面要学到的类都是大量存在的，这些变量、函数和类的名称将都存在于全局作用域中，可能会导致很多冲突。使用命名空间的目的是对标识符的名称进行本地化，以避免命名冲突或名字污染，namespace关键字的出现就是针对这种问题的。

c语言项目类似下面程序这样的命名冲突是普遍存在的问题，C++引入namespace就是为了更好的解决这样的问题

```

1 #include <stdio.h>

                                #include <stdio.h>

2 #include <stdlib.h>
3 int rand = 10;
4
5 int main()
6 {
7     // 编译报错: error C2365: "rand": 重定义; 以前的定义是“函数”
8     printf("%d\n", rand);
9     return 0;
10 }

```

7.2 namespace的定义

- 定义命名空间，需要使用到namespace关键字，后面跟命名空间的名字，然后接一对{}即可，{}中即为命名空间的成员。命名空间中可以定义变量/函数/类型等。
- namespace本质是定义出一个域，这个域跟全局域各自独立，不同的域可以定义同名变量，所以下面的rand不在冲突了。
- C++中域有函数局部域，全局域，命名空间域，类域；域影响的是编译时语法查找一个变量/函数/类型出处(声明或定义)的逻辑，所有有了域隔离，名字冲突就解决了。局部域和全局域除了会影响编译查找逻辑，还会影响变量的生命周期，命名空间域和类域不影响变量生命周期。

- namespace只能定义在全局，当然他还可以嵌套定义。
- 项目工程中多文件中定义的同名namespace会认为是一个namespace，不会冲突。
- C++标准库都放在一个叫std(standard)的命名空间中。

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // 1. 正常的命名空间定义
5
6  // bit是命名空间的名字，一般开发中是用项目名字做命名空间名。
7  // 我们上课用的是bit，大家下去以后自己练习可以考虑用自己名字缩写，如张三：zs
8  namespace bit
9  {
10     // 命名空间中可以定义变量/函数/类型
11     int rand = 10;
12
13     int Add(int left, int right)
14     {
15         return left + right;
16     }
17
18     struct Node
19     {
20         struct Node* next;
21         int val;
22     };
23 }
24
25 int main()
26 {
27     // 这里默认是访问的是全局的rand函数指针
28     printf("%p\n", rand);
29     // 这里指定bit命名空间中的rand
30     printf("%d\n", bit::rand);
31
32     return 0;
33 }

```

```

1  //2. 命名空间可以嵌套
2  namespace bit
3  {
4      // 鹏哥
5      namespace pg

```

```

6      {
7          int rand = 1;
8
9          int Add(int left, int right)
10         {
11             return left + right;
12         }
13     }
14
15     // 杭哥
16     namespace hg
17     {
18         int rand = 2;
19
20         int Add(int left, int right)
21         {
22             return (left + right)*10;
23         }
24     }
25 }
26
27 int main()
28 {
29     printf("%d\n", bit::pg::rand);
30     printf("%d\n", bit::hg::rand);
31
32     printf("%d\n", bit::pg::Add(1, 2));
33     printf("%d\n", bit::hg::Add(1, 2));
34
35     return 0;
36 }

```

```

1  // 多文件中可以定义同名namespace, 他们会默认合并到一起, 就像同一个namespace一样
2  // Stack.h
3  #pragma once
4  #include<stdio.h>
5  #include<stdlib.h>
6  #include<stdbool.h>
7  #include<assert.h>
8
9  namespace bit
10 {
11     typedef int STDataType;
12     typedef struct Stack
13     {

```

```

14         STDataType* a;
15         int top;
16         int capacity;
17     }ST;
18
19     void STInit(ST* ps, int n);
20     void STDestroy(ST* ps);
21
22     void STPush(ST* ps, STDataType x);
23     void STPop(ST* ps);
24     STDataType STTop(ST* ps);
25     int STSize(ST* ps);
26     bool STEmpty(ST* ps);
27 }
28
29 // Stack.cpp
30 #include "Stack.h"
31 namespace bit
32 {
33     void STInit(ST* ps, int n)
34     {
35         assert(ps);
36
37         ps->a = (STDataType*)malloc(n * sizeof(STDataType));
38         ps->top = 0;
39         ps->capacity = n;
40     }
41
42     // 栈顶
43     void STPush(ST* ps, STDataType x)
44     {
45         assert(ps);
46
47         // 满了, 扩容
48         if (ps->top == ps->capacity)
49         {
50             printf("扩容\n");
51             int newcapacity = ps->capacity == 0 ? 4 : ps->capacity
52 * 2;
53             STDataType* tmp = (STDataType*)realloc(ps->a,
54 newcapacity * sizeof(STDataType));
55             if (tmp == NULL)
56             {
57                 perror("realloc fail");
58                 return;
59             }
60         }
61     }
62
63     void STPop(ST* ps)
64     {
65         assert(ps);
66         assert(ps->top > 0);
67         ps->top--;
68     }
69
70     STDataType STTop(ST* ps)
71     {
72         assert(ps);
73         assert(ps->top >= 0);
74         return ps->a[ps->top];
75     }
76
77     int STSize(ST* ps)
78     {
79         assert(ps);
80         return ps->top + 1;
81     }
82
83     bool STEmpty(ST* ps)
84     {
85         assert(ps);
86         return ps->top < 0;
87     }
88
89     void STDestroy(ST* ps)
90     {
91         assert(ps);
92         free(ps->a);
93         ps->a = NULL;
94     }
95 }

```

```

59             ps->a = tmp;
60             ps->capacity = newcapacity;
61         }
62
63         ps->a[ps->top] = x;
64         ps->top++;
65     }
66
67     //...
68 }
69
70 // Queue.h
71 #pragma once
72 #include<stdlib.h>
73 #include<stdbool.h>
74 #include<assert.h>
75
76 namespace bit
77 {
78     typedef int QDataType;
79     typedef struct QueueNode
80     {
81         int val;
82         struct QueueNode* next;
83     }QNode;
84
85     typedef struct Queue
86     {
87         QNode* phead;
88         QNode* ptail;
89         int size;
90     }Queue;
91
92     void QueueInit(Queue* pq);
93     void QueueDestroy(Queue* pq);
94
95     // 入队列
96     void QueuePush(Queue* pq, QDataType x);
97     // 出队列
98     void QueuePop(Queue* pq);
99
100     QDataType QueueFront(Queue* pq);
101     QDataType QueueBack(Queue* pq);
102     bool QueueEmpty(Queue* pq);
103     int QueueSize(Queue* pq);
104 }
105

```

```
106 // Queue.cpp
107 #include"Queue.h"
108 namespace bit
109 {
110     void QueueInit(Queue* pq)
111     {
112         assert(pq);
113
114         pq->phead = NULL;
115         pq->ptail = NULL;
116         pq->size = 0;
117     }
118
119     // ...
120 }
121
122 // test.cpp
123 #include"Queue.h"
124 #include"Stack.h"
125
126 // 全局定义了一份单独的Stack
127 typedef struct Stack
128 {
129     int a[10];
130     int top;
131 }ST;
132
133 void STInit(ST* ps){}
134 void STPush(ST* ps, int x){}
135
136 int main()
137 {
138     // 调用全局的
139     ST st1;
140     STInit(&st1);
141     STPush(&st1, 1);
142     STPush(&st1, 2);
143     printf("%d\n", sizeof(st1));
144
145     // 调用bit namespace的
146     bit::ST st2;
147     printf("%d\n", sizeof(st2));
148     bit::STInit(&st2);
149     bit::STPush(&st2, 1);
150     bit::STPush(&st2, 2);
151
152     return 0;
```

7.3 命名空间使用

编译查找一个变量的声明/定义时，默认只会在局部或者全局查找，不会到命名空间里面去查找。所以下面程序会编译报错。所以我们要使用命名空间中定义的变量/函数，有三种方式：

- 指定命名空间访问，项目中推荐这种方式。
- using将命名空间中某个成员展开，项目中经常访问的不存在冲突的成员推荐这种方式。
- 展开命名空间中全部成员，项目不推荐，冲突风险很大，日常小练习程序为了方便推荐使用。

```

1  #include<stdio.h>
2
3  namespace bit
4  {
5      int a = 0;
6      int b = 1;
7  }
8
9  int main()
10 {
11     // 编译报错: error C2065: "a": 未声明的标识符
12     printf("%d\n", a);
13     return 0;
14 }
```

```

1  // 指定命名空间访问
2  int main()
3  {
4      printf("%d\n", N::a);
5      return 0;
6  }
7
8  // using将命名空间中某个成员展开
9  using N::b;
10 int main()
11 {
12     printf("%d\n", N::a);
13     printf("%d\n", b);
14     return 0;
15 }
16
```

```

17 // 展开命名空间中全部成员
18 using namespace N;
19 int main()
20 {
21     printf("%d\n", a);
22     printf("%d\n", b);
23     return 0;
24 }
25

```

8. C++输入&输出

- <iostream> 是 Input Output Stream 的缩写，是标准的输入、输出流库，定义了标准的输入、输出对象。
- std::cin 是 istream 类的对象，它主要面向窄字符（narrow characters (of type char)）的标准输入流。
- std::cout 是 ostream 类的对象，它主要面向窄字符的标准输出流。
- std::endl 是一个函数，流插入输出时，相当于插入一个换行字符加刷新缓冲区。
- <<是流插入运算符，>>是流提取运算符。（C语言还用这两个运算符做位运算左移/右移）
- 使用C++输入输出更方便，不需要像printf/scanf输入输出时那样，需要手动指定格式，C++的输入输出可以自动识别变量类型(本质是通过函数重载实现的，这个以后会讲到)，其实最重要的是C++的流能更好的支持自定义类型对象的输入输出。
- IO流涉及类和对象，运算符重载、继承等很多面向对象的知识，这些知识我们还没有讲解，所以这里我们只能简单认识一下C++ IO流的使用，后面我们会有专门的一个章节来细节IO流库。
- cout/cin/endl等都属于C++标准库，C++标准库都放在一个叫std(standard)的命名空间中，所以要通过命名空间的使用方式去用他们。
- 一般日常练习中我们可以using namespace std，实际项目开发中不建议using namespace std。
- 这里我们没有包含<stdio.h>，也可以使用printf和scanf，在包含<iostream>间接包含了。vs系列编译器是这样的，其他编译器可能会报错。

```

1 #define _CRT_SECURE_NO_WARNINGS 1
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int a = 0;
8     double b = 0.1;
9     char c = 'x';

```

```

10
11     cout << a << " " << b << " " << c << endl;
12     std::cout << a << " " << b << " " << c << std::endl;
13
14     scanf("%d%lf", &a, &b);
15     printf("%d %lf\n", a, b);
16
17     // 可以自动识别变量的类型
18     cin >> a;
19     cin >> b >> c;
20
21     cout << a << endl;
22     cout << b << " " << c << endl;
23
24     return 0;
25 }

```

```

1 #include<iostream>
2 using namespace std;
3
4 int main()
5 {
6     // 在io需求比较高的地方，如部分大量输入的竞赛题中，加上以下3行代码
7     // 可以提高C++IO效率
8     ios_base::sync_with_stdio(false);
9     cin.tie(nullptr);
10    cout.tie(nullptr);
11
12    return 0;
13 }

```

9. 缺省参数

- 缺省参数是声明或定义函数时为函数的参数指定一个缺省值。在调用该函数时，如果没有指定实参则采用该形参的缺省值，否则使用指定的实参，缺省参数分为全缺省和半缺省参数。（有些地方把缺省参数也叫默认参数）
- 全缺省就是全部形参给缺省值，半缺省就是部分形参给缺省值。C++规定半缺省参数必须从右往左依次连续缺省，不能间隔跳跃给缺省值。
- 带缺省参数的函数调用，C++规定必须从左到右依次给实参，不能跳跃给实参。
- 函数声明和定义分离时，缺省参数不能在函数声明和定义中同时出现，规定必须函数声明给缺省值。

```

1 #include <iostream>
2 #include <assert.h>
3 using namespace std;
4
5 void Func(int a = 0)
6 {
7     cout << a << endl;
8 }
9
10 int main()
11 {
12     Func();      // 没有传参时，使用参数的默认值
13     Func(10);    // 传参时，使用指定的实参
14
15     return 0;
16 }

```

```

1 #include <iostream>
2 using namespace std;
3
4 // 全缺省
5 void Func1(int a = 10, int b = 20, int c = 30)
6 {
7     cout << "a = " << a << endl;
8     cout << "b = " << b << endl;
9     cout << "c = " << c << endl << endl;
10 }
11
12 // 半缺省
13 void Func2(int a, int b = 10, int c = 20)
14 {
15     cout << "a = " << a << endl;
16     cout << "b = " << b << endl;
17     cout << "c = " << c << endl << endl;
18 }
19
20 int main()
21 {
22     Func1();
23     Func1(1);
24     Func1(1,2);
25     Func1(1,2,3);
26
27     Func2(100);
28     Func2(100, 200);

```

```
29     Func2(100, 200, 300);
30
31     return 0;
32 }
```

```
1 // Stack.h
2 #include <iostream>
3 #include <assert.h>
4 using namespace std;
5
6 typedef int STDataType;
7 typedef struct Stack
8 {
9     STDataType* a;
10    int top;
11    int capacity;
12 }ST;
13 void STInit(ST* ps, int n = 4);
14
15 // Stack.cpp
16 #include "Stack.h"
17 // 缺省参数不能声明和定义同时给
18 void STInit(ST* ps, int n)
19 {
20     assert(ps && n > 0);
21
22     ps->a = (STDataType*)malloc(n * sizeof(STDataType));
23     ps->top = 0;
24     ps->capacity = n;
25 }
26
27 // test.cpp
28 #include "Stack.h"
29 int main()
30 {
31     ST s1;
32     STInit(&s1);
33
34     // 确定知道要插入1000个数据，初始化时一把开好，避免扩容
35     ST s2;
36     STInit(&s2, 1000);
37
38     return 0;
39 }
```

10. 函数重载

C++支持在同一作用域中出现同名函数，但是要求这些同名函数的形参不同，可以是参数个数不同或者类型不同。这样C++函数调用就表现出了多态行为，使用更灵活。C语言是不支持同一作用域中出现同名函数的。

```
1  #include<iostream>
2  using namespace std;
3
4  // 1、参数类型不同
5  int Add(int left, int right)
6  {
7      cout << "int Add(int left, int right)" << endl;
8
9      return left + right;
10 }
11
12 double Add(double left, double right)
13 {
14     cout << "double Add(double left, double right)" << endl;
15
16     return left + right;
17 }
18
19 // 2、参数个数不同
20 void f()
21 {
22     cout << "f()" << endl;
23 }
24
25 void f(int a)
26 {
27     cout << "f(int a)" << endl;
28 }
29
30 // 3、参数类型顺序不同
31 void f(int a, char b)
32 {
33     cout << "f(int a,char b)" << endl;
34 }
35
36 void f(char b, int a)
37 {
38     cout << "f(char b, int a)" << endl;
39 }
40
```

```

41 // 返回值不同不能作为重载条件，因为调用时也无法区分
42 //void fxx()
43 //{
44 //
45 //int fxx()
46 //{
47 //    return 0;
48 //}
49
50 // 下面两个函数构成重载
51 // f()但是调用时，会报错，存在歧义，编译器不知道调用谁
52 void f1()
53 {
54     cout << "f()" << endl;
55 }
56
57 void f1(int a = 10)
58 {
59     cout << "f(int a)" << endl;
60 }
61
62 int main()
63 {
64     Add(10, 20);
65     Add(10.1, 20.2);
66
67     f();
68     f(10);
69
70     f(10, 'a');
71     f('a', 10);
72
73     return 0;
74 }

```

11. 引用

11.1 引用的概念和定义

引用不是新定义一个变量，而是给已存在变量取了一个别名，编译器不会为引用变量开辟内存空间，它和它引用的变量共用同一块内存空间。比如：水壶传中李逵，宋江叫"铁牛"，江湖上人称"黑旋风"；林冲，外号豹子头；

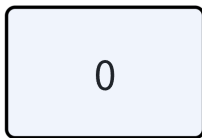
类型& 引用别名 = 引用对象;

C++中为了避免引入太多的运算符，会复用C语言的一些符号，比如前面的<< 和 >>，这里引用也和取地址使用了同一个符号&，大家注意使用方法角度区分就可以。（吐槽一下，这个问题其实挺坑的，个人觉得用更多符号反而更好，不容易混淆）



```
1 #include<iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a = 0;
7
8     // 引用：b和c是a的别名
9     int& b = a;
10    int& c = a;
11    // 也可以给别名b取别名，d相当于还是a的别名
12    int& d = b;
13
14    ++d;
15
16    // 这里取地址我们看到是一样的
17    cout << &a << endl;
18    cout << &b << endl;
19    cout << &c << endl;
20    cout << &d << endl;
21
22    return 0;
23 }
```

a
b
c
d



11.2 引用的特性

- 引用在定义时必须初始化
- 一个变量可以有多个引用
- 引用一旦引用一个实体，再不能引用其他实体

```
1 #include<iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a = 10;
7
8     // 编译报错：“ra”：必须初始化引用
9     //int& ra;
10
11    int& b = a;
12
13    int c = 20;
14    // 这里并非让b引用c，因为C++引用不能改变指向，
15    // 这里是一个赋值
16    b = c;
17
18    cout << &a << endl;
19    cout << &b << endl;
20    cout << &c << endl;
21
22    return 0;
23 }
```

11.3 引用的使用

- 引用在实践中主要是于引用传参和引用做返回值中减少拷贝提高效率和改变引用对象时同时改变被引用对象。

- 引用传参跟指针传参功能是类似的，引用传参相对更方便一些。
- 引用返回值的场景相对比较复杂，我们在这里简单讲了一下场景，还有一些内容后续类和对象章节中会继续深入讲解。
- 引用和指针在实践中相辅相成，功能有重叠性，但是各有特点，互相不可替代。C++的引用跟其他语言的引用(如Java)是有很大的区别的，除了用法，最大的点，C++引用定义后不能改变指向，Java的引用可以改变指向。
- 一些主要用C代码实现版本数据结构教材中，使用C++引用替代指针传参，目的是简化程序，避开复杂的指针，但是很多同学没学过引用，导致一头雾水。

```
1 void Swap(int& rx, int& ry)
2 {
3     int tmp = rx;
4     rx = ry;
5     ry = tmp;
6 }
7
8 int main()
9 {
10     int x = 0, y = 1;
11     cout << x << " " << y << endl;
12
13     Swap(x, y);
14     cout << x << " " << y << endl;
15
16     return 0;
17 }
```

```
1 #include<iostream>
2 using namespace std;
3
4 typedef int STDataType;
5 typedef struct Stack
6 {
7     STDataType* a;
8     int top;
9     int capacity;
10 }ST;
11
12 void STInit(ST& rs, int n = 4)
13 {
14     rs.a = (STDataType*)malloc(n * sizeof(STDataType));
15     rs.top = 0;
```

```

16     rs.capacity = n;
17 }
18
19 // 栈顶
20 void STPush(ST& rs, STDataType x)
21 {
22     assert(ps);
23
24     // 满了, 扩容
25     if (rs.top == rs.capacity)
26     {
27         printf("扩容\n");
28         int newcapacity = rs.capacity == 0 ? 4 : rs.capacity * 2;
29         STDataType* tmp = (STDataType*)realloc(rs.a, newcapacity *
sizeof(STDataType));
30         if (tmp == NULL)
31         {
32             perror("realloc fail");
33             return;
34         }
35
36         rs.a = tmp;
37         rs.capacity = newcapacity;
38     }
39
40     rs.a[rs.top] = x;
41     rs.top++;
42 }
43
44 // int STTop(ST& rs)
45 int& STTop(ST& rs)
46 {
47     assert(rs.top > 0);
48
49     return rs.a[rs.top];
50 }
51
52 int main()
53 {
54     // 调用全局的
55     ST st1;
56     STInit(st1);
57     STPush(st1, 1);
58     STPush(st1, 2);
59
60     cout << STTop(st1) << endl;
61

```

```

62     STTop(st1) += 10;
63
64     cout << STTop(st1) << endl;
65
66     return 0;
67 }

```

```

1  #include<iostream>
2  using namespace std;
3
4  typedef struct SeqList
5  {
6      int a[10];
7      int size;
8  }SLT;
9
10 // 一些主要用C代码实现版本数据结构教材中，使用C++引用替代指针传参，目的是简化程序，避开复
    杂的指针，但是很多同学没学过引用，导致一头雾水。
11 void SeqPushBack(SLT& sl, int x)
12 {}
13
14 typedef struct ListNode
15 {
16     int val;
17     struct ListNode* next;
18 }LTNode, *PNode;
19
20 // 指针变量也可以取别名，这里LTNode*& phead就是给指针变量取别名
21 // 这样就不需要用二级指针了，相对而言简化了程序
22 //void ListPushBack(LTNode** phead, int x)
23 //void ListPushBack(LTNode*& phead, int x)
24 void ListPushBack(PNode& phead, int x)
25 {
26     PNode newnode = (PNode)malloc(sizeof(LTNode));
27     newnode->val = x;
28     newnode->next = NULL;
29     if (phead == NULL)
30     {
31         phead = newnode;
32     }
33     else
34     {
35         //...
36     }
37 }

```

```

38
39 int main()
40 {
41     PNode plist = NULL;
42     ListPushBack(plist, 1);
43
44     return 0;
45 }

```

11.4 const引用

- 可以引用一个const对象，但是必须用const引用。const引用也可以引用普通对象，因为对象的访问权限在引用过程中可以缩小，但是不能放大。
- 需要注意的是类似 `int& rb = a*3; double d = 12.34; int& rd = d;` 这样一些场景下 `a*3` 的结果保存在一个临时对象中，`int& rd = d` 也是类似，在类型转换中会产生临时对象存储中间值，也就是时，`rb`和`rd`引用的都是临时对象，而C++规定临时对象具有常性，所以这里就触发了权限放大，必须要用常引用才可以。
- 所谓临时对象就是编译器需要一个空间暂存表达式的求值结果时临时创建的一个未命名的对象，C++中把这个未命名对象叫做临时对象。

```

1 int main()
2 {
3     const int a = 10;
4     // 编译报错: error C2440: “初始化”: 无法从“const int”转换为“int &”
5     // 这里的引用是对a访问权限的放大
6     //int& ra = a;
7
8     // 这样才可以
9     const int& ra = a;
10    // 编译报错: error C3892: “ra”: 不能给常量赋值
11    //ra++;
12
13    // 这里的引用是对b访问权限的缩小
14    int b = 20;
15    const int& rb = b;
16    // 编译报错: error C3892: “rb”: 不能给常量赋值
17    //rb++;
18
19    return 0;
20 }

```

```

1 #include<iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a = 10;
7
8     const int& ra = 30;
9
10    // 编译报错：“初始化”：无法从“int”转换为“int &”
11    // int& rb = a * 3;
12    const int& rb = a*3;
13
14    double d = 12.34;
15    // 编译报错：“初始化”：无法从“double”转换为“int &”
16    // int& rd = d;
17
18    const int& rd = d;
19
20    return 0;
21 }

```

11.5 指针和引用的关系

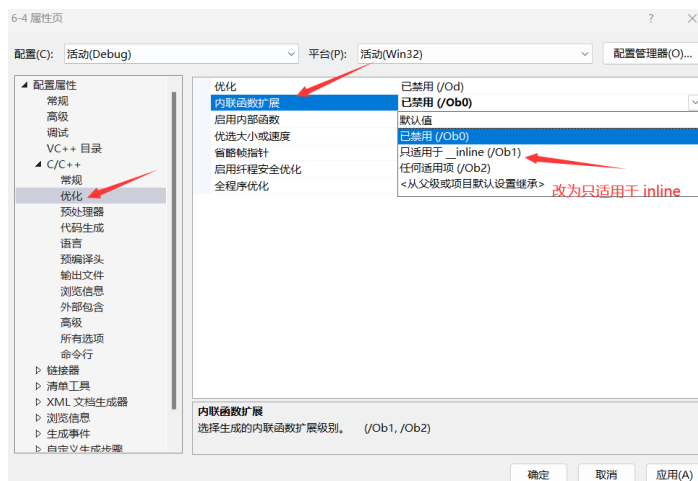
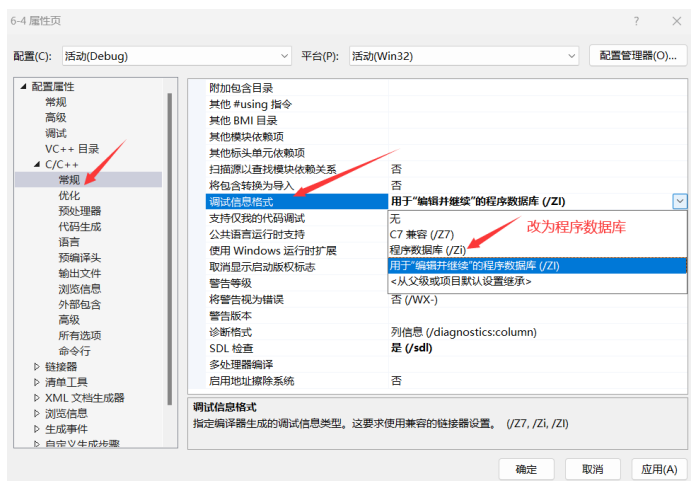
C++中指针和引用就像两个性格迥异的亲兄弟，指针是哥哥，引用是弟弟，在实践中他们相辅相成，功能有重叠性，但是各有自己的特点，互相不可替代。

- 语法概念上引用是一个变量的取别名不开空间，指针是存储一个变量地址，要开空间。
- 引用在定义时必须初始化，指针建议初始化，但是语法上不是必须的。
- 引用在初始化时引用一个对象后，就不能再引用其他对象；而指针可以在不断地改变指向对象。
- 引用可以直接访问指向对象，指针需要解引用才是访问指向对象。
- sizeof中含义不同，引用结果为引用类型的大小，但指针始终是地址空间所占字节个数(32位平台下占4个字节，64位下是8byte)
- 指针很容易出现空指针和野指针的问题，引用很少出现，引用使用起来相对更安全一些。

12. inline

- 用inline修饰的函数叫做内联函数，编译时C++编译器会在调用的地方展开内联函数，这样调用内联函数就需要建立栈帧了，就可以提高效率。
- inline对于编译器而言只是一个建议，也就是说，你加了inline编译器也可以选择不在调用的地方不展开，不同编译器关于inline什么情况展开各不相同，因为C++标准没有规定这个。inline适用于频繁调用的短小函数，对于递归函数，代码相对多一些的函数，加上inline也会被编译器忽略。

- C语言实现宏函数也会在预处理时替换展开，但是宏函数实现很复杂很容易出错的，且不方便调试，C++设计了inline目的就是替代C的宏函数。
- vs编译器 debug版本下面默认是不展开inline的，这样方便调试，debug版本想展开需要设置一下以下两个地方。
- inline不建议声明和定义分离到两个文件，分离会导致链接错误。因为inline被展开，就没有函数地址，链接时会出现报错。



```

1  #include<iostream>
2  using namespace std;
3
4  inline int Add(int x, int y)
5  {
6      int ret = x + y;
7      ret += 1;
8      ret += 1;
9      ret += 1;
10
11     return ret;
12 }
13
14 int main()
15 {
16     // 可以通过汇编观察程序是否展开
17     // 有call Add语句就是没有展开，没有就是展开了
18     int ret = Add(1, 2);
19     cout << Add(1, 2) * 5 << endl;
20
21     return 0;
22 }

```

```

1 #include<iostream>
2 using namespace std;
3
4 // 实现一个ADD宏函数的常见问题
5 // #define ADD(int a, int b) return a + b;
6 // #define ADD(a, b) a + b;
7 // #define ADD(a, b) (a + b)
8
9 // 正确的宏实现
10 #define ADD(a, b) ((a) + (b))
11 // 为什么不能加分号?
12 // 为什么要加外面的括号?
13 // 为什么要加里面的括号?
14
15 int main()
16 {
17     int ret = ADD(1, 2);
18
19     cout << ADD(1, 2) << endl;
20
21     cout << ADD(1, 2)*5 << endl;
22
23     int x = 1, y = 2;
24     ADD(x & y, x | y); // -> (x&y+x|y)
25
26     return 0;
27 }

```

```

1 // F.h
2 #include <iostream>
3 using namespace std;
4
5 inline void f(int i);
6
7 // F.cpp
8 #include "F.h"
9 void f(int i)
10 {
11     cout << i << endl;
12 }
13
14 // main.cpp
15 #include "F.h"
16 int main()
17 {

```

```
18 // 链接错误: 无法解析的外部符号 "void __cdecl f(int)" (?f@@YAXH@Z)
19 f(10);
20 return 0;
21 }
```

13. nullptr

NULL实际是一个宏，在传统的C头文件(stddef.h)中，可以看到如下代码：

```
1 #ifndef NULL
2     #ifdef __cplusplus
3         #define NULL    0
4     #else
5         #define NULL    ((void *)0)
6     #endif
7 #endif
```

- C++中NULL可能被定义为字面常量0，或者C中被定义为无类型指针(void*)的常量。不论采取何种定义，在使用空值的指针时，都不可避免的会遇到一些麻烦，本想通过f(NULL)调用指针版本的f(int*)函数，但是由于NULL被定义成0，调用了f(int x)，因此与程序的初衷相悖。f((void*)NULL);调用会报错。
- C++11中引入nullptr，nullptr是一个特殊的关键字，nullptr是一种特殊类型的字面量，它可以转换成任意其他类型的指针类型。使用nullptr定义空指针可以避免类型转换的问题，因为nullptr只能被隐式地转换为指针类型，而不能被转换为整数类型。

```
1 #include<iostream>
2 using namespace std;
3
4 void f(int x)
5 {
6     cout << "f(int x)" << endl;
7 }
8
9 void f(int* ptr)
10 {
11     cout << "f(int* ptr)" << endl;
12 }
13
14 int main()
15 {
16     f(0);
```

```
17 // 本想通过f(NULL)调用指针版本的f(int*)函数，但是由于NULL被定义成0，调用了f(int
    x)，因此与程序的初衷相悖。
18 f(NULL);
19 f((int*)NULL);
20
21 // 编译报错: error C2665: “f”: 2 个重载中没有一个可以转换所有参数类型
22 // f((void*)NULL);
23
24 f(nullptr);
25
26 return 0;
27 }
```